

ModiJointCanevo 接口文档

目录

- 1. 概述
- 2. 数据类型
 - 2.1 CanEvoError — 错误码枚举
 - 2.2 CanEvoMode — 工作模式枚举
 - 2.3 CanEvoServoState — 伺服状态机枚举
 - 2.4 CanEvoFault — 故障码枚举
 - 2.5 CanEvoWarn — 警告码枚举
 - 2.6 JointStatus — 关节状态结构体
- 3. 常量定义
- 4. modi_bus_canevo 接口列表
 - 4.1 构造与生命周期
 - 4.2 总线控制
 - 4.3 超时配置
- 5. modi_joint_canevo 接口列表
 - 5.1 构造与生命周期
 - 5.2 PDO 实时控制（非阻塞）
 - 5.3 状态读取（非阻塞）
 - 5.4 控制字控制（非阻塞）
 - 5.5 状态查询辅助（非阻塞）
 - 5.6 SDO 设备信息（阻塞，Index 0x00）
 - 5.7 SDO 关节参数（阻塞，Index 0x01）
 - 5.8 SDO 控制器参数（阻塞，Index 0x07）
 - 5.9 SDO 编码器参数（阻塞，Index 0x08）
 - 5.10 SDO PP 模式轮廓参数（阻塞，Index 0x21）
 - 5.11 SDO 关节实际状态（阻塞，Index 0x20，只读）
 - 5.12 SDO 控制字操作（阻塞）
- 6. PDO 映射方案
- 7. 对象字典索引参考
- 8. 典型用法
- 9. 注意事项

1. 概述

`modi_bus_canevo` 和 `modi_joint_canevo` 是基于 CanEvo V1.2.3 CAN-FD 协议的 x86 (Linux/SocketCAN) 平台关节控制 C++ SDK。

采用 **Bus + Joint** 两级架构：

- `modi_bus_canevo`：总线管理类，对应一条 SocketCAN 接口（如 can0），负责收发线程管理和 SYNC 广播
- `modi_joint_canevo`：关节控制类，对应总线上一个节点（node_id 1~62），负责 PDO 实时控制和 SDO 配置诊断

所有接口分为两类：

- **Rt 接口（实时接口，Real-time）**：以 `Rt` 前缀命名，包括 PDO 控制、状态读取、控制字控制等。这些接口**非阻塞**，通过 SPSC 无锁队列 + 专用 Tx 线程发送，保证实时性，适用于高频控制循环（建议 200Hz~1000Hz）。典型接口：`RtSetCspTargetPosition()`、`RtGetJointStatus()`、`RtEnable()`、`RtSendSync()` 等。
- **Nrt 接口（非实时接口，Non-real-time）**：以 `Nrt` 前缀命名，包括生命周期管理、SDO 配置诊断等。SDO 接口为**同步阻塞调用**，会等待从站应答或超时（默认 100ms），**不应在实时控制循环中调用**。典型接口：`NrtInit()`、

`NrtDestroy()`、`NrtGetProtocolVersion()`、`NrtSetMaxSpeed()` 等。

接口命名规则：

- **Rt** 前缀 = Real-time（实时），用于 PDO 相关操作，非阻塞，适合实时循环
- **Nrt** 前缀 = Non-real-time（非实时），用于 SDO 相关操作，阻塞调用，适合初始化/配置阶段

支持的工作模式：CSP(1)、CSV(2)、CST(3)、PP(4)

公共接口单位约定：

- 位置：**rad**（弧度）
- 速度：**rad/s**
- 加速度/减速度：**rad/s²**
- 电流：**A**（安培）
- 温度：**°C**
- 电压：**V**

SDK 内部自动完成 rad↔deg、rad/s↔rpm 等单位转换，用户无需关心协议层单位。

2. 数据类型

2.1 CanEvoError — 错误码枚举

SDK 接口返回值统一使用的错误码，底层类型 `int`。

枚举名	值	说明
<code>kOk</code>	0	成功
<code>kInvalidParam</code>	-1	参数非法（空指针/越界/不合理组合）
<code>kBusNotOpen</code>	-2	总线未打开
<code>kSendFailed</code>	-3	CAN 发送失败
<code>kSdoTimeout</code>	-4	SDO 应答超时
<code>kSdoAbort</code>	-5	SDO 应答式故障（从站返回错误）
<code>kNotInitialized</code>	-6	Joint 未初始化
<code>kNodeNotFound</code>	-7	节点未注册
<code>kQueueFull</code>	-8	发送队列满

2.2 CanEvoMode — 工作模式枚举

控制字 bit12~bit15 编码的工作模式，底层类型 `uint8_t`。

枚举名	值	说明
<code>kCsp</code>	1	周期同步位置模式 (Cyclic Synchronous Position)
<code>kCsv</code>	2	周期同步速度模式 (Cyclic Synchronous Velocity)
<code>kCst</code>	3	周期同步转矩模式 (Cyclic Synchronous Torque)
<code>kPp</code>	4	轮廓位置模式 (Profile Position)

2.3 CanEvoServoState — 伺服状态机枚举

状态字 bit0~bit3 编码的伺服状态机状态，底层类型 `uint8_t`。

枚举名	值	说明
kInit	0x00	初始化
kDisabled	0x01	禁止使能
kReady	0x02	准备使能
kRunning	0x03	运行（伺服使能）
kQuickStop	0x04	快速停机
kFaultReact	0x06	故障响应
kFault	0x07	故障
kInitFault	0x08	初始化故障

2.4 CanEvoFault — 故障码枚举

对象字典 0x0008 故障码定义，底层类型 uint16_t。同时用于 EMCY 紧急报文。

枚举名	值	说明
kNone	0x0000	无故障
kOverCurrent	0x2130	相电流过流
kBusOverVolt	0x3210	总线过压
kBusUnderVolt	0x3220	总线欠压
kMotorOverload	0x3230	电机过载
kPhaseLoss	0x3331	电机动力线缺相
kOverTemp	0x4310	关节过温保护
kUnderTemp	0x4320	关节低温保护
kParamStorage	0x5520	硬件参数存储错误
kBrake	0x7110	抱闸异常
kMotorStall	0x7121	电机堵转保护
kEncoder1	0x7303	编码器1错误
kEncoder2	0x7304	编码器2错误
kEnc1Battery	0x7385	多圈编码器1电池低电压
kEnc2Battery	0x7386	多圈编码器2电池低电压
kComm	0x8100	通讯错误
kOverSpeed	0x8400	超过最大速度
kFollowError	0x8611	跟随误差过大
kCurrentSensor	0xFF00	电流传感器故障
kEncZLoss	0xFF01	编码器Z信号丢失
kPeakOverload	0xFF02	关节峰值电流过载
kHwShort	0xFF03	硬件短路保护
kHall	0xFF04	编码器Hall信号故障
kSyncError	0xFF05	同步周期误差过大
kAngleIdent	0xFF06	电角度辨识错误

枚举名	值	说明
kTimeout	0xFF07	伺服程序运行超时
kFwError	0xFF08	固件错误

2.5 CanEvoWarn — 警告码枚举

对象字典 0x0009 警告码定义，底层类型 `uint16_t`。

枚举名	值	说明
kNone	0x0000	无警告
kPcbHighTemp	0x2101	PCB高温
kMotorHighTemp	0x2102	电机高温
kGearHighTemp	0x2103	减速器高温
kPhaseOverload	0x2201	相电流过载
kSpeedHigh	0x2301	速度过高
kPosLimit	0x2401	位置超过限制值
kFlashErase	0x2601	Flash擦除次数过高
kVoltHigh	0x2701	供电电压高
kVoltLow	0x2702	供电电压低

2.6 JointStatus — 关节状态结构体

存储从 TxPDO0 解析出的关节状态数据，所有物理量均为工程单位。

字段名	类型	单位	说明
statusword	uint16_t	—	原始状态字（16bit）
actual_pos_rad	float	rad	实际位置
actual_vel_rads	float	rad/s	实际速度
actual_cur_a	float	A	实际转矩电流
actual_acc_radss	float	rad/s ²	实际加速度
bus_voltage_v	float	V	母线电压（raw×0.01）
pcb_temp_c	float	°C	PCB温度（raw×0.1）
motor_temp_c	float	°C	电机温度（raw×0.1）

3. 常量定义

常量名	值	说明
kCanevoBrakeOpen	0xAA55	抱闸打开控制字
kCanevoBrakeClose	0xBB66	抱闸关闭控制字

4. modi_bus_canevo 接口列表

代表一条 SocketCAN 总线（如 can0），负责管理 SocketCAN socket 生命周期、收包分发、SYNC 广播和 PDO 发送队列。多个 Joint 共享同一个 Bus 实例。

4.1 构造与生命周期

序号	接口名字	参数	返回值	说明
1	<code>modi_bus_canevo()</code>	无	—	构造函数
2	<code>~modi_bus_canevo()</code>	无	—	析构函数

禁止拷贝构造和拷贝赋值。

4.2 总线控制

序号	接口名字	参数	返回值	说明
3	<code>Open(can_ifname)</code>	<code>can_ifname</code> : SocketCAN 接口名 (const std::string&)，如 "can0"	<code>int</code> — CanEvoError::kOk 成功	打开 SocketCAN 总线，创建收/发线程
4	<code>Close()</code>	无	<code>void</code>	关闭总线，停止线程，释放所有资源
5	<code>IsOpen()</code>	无	<code>bool</code> — <code>true</code> : 已打开	查询总线是否已打开
6	<code>RtSendSync(counter)</code>	<code>counter</code> : 同步计数器 (const uint8_t, 0~255 循环递增)	<code>int</code> — CanEvoError::kOk 成功	发送 SYNC 广播帧 (cob_id=0x03F)，每个控制周期调用一次。使用独立 socket fd 发送，避免与 PDO 争抢锁 [实时接口]

4.3 超时配置

序号	接口名字	参数	返回值	说明
7	<code>SetSdoTimeoutMs(ms)</code>	<code>ms</code> : 超时毫秒数 (const int)	<code>void</code>	设置 SDO 默认超时（作为后续创建 joint 的默认值）
8	<code>SetPdoTimeoutMs(ms)</code>	<code>ms</code> : 超时毫秒数 (const int)	<code>void</code>	设置 PDO 默认超时

5. modi_joint_canevo 接口列表

代表总线上的一个关节节点（node_id 1~62），负责 PDO 实时控制和 SDO 配置诊断。

5.1 构造与生命周期

序号	接口名字	参数	返回值	说明
1	<code>modi_joint_canevo()</code>	无	—	构造函数
2	<code>~modi_joint_canevo()</code>	无	—	析构函数（自动调用 NrtDestroy）

序号	接口名字	参数	返回值	说明
3	<code>NrtInit(bus, node_id)</code>	<code>bus</code> : 已打开的总线实例引用 <code>(modi_bus_canevo&)</code> <code>node_id</code> : 节点 ID <code>(const uint8_t, 1~62)</code>	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kBusNotOpen</code> 总线未打开； <code>kInvalidParam</code> 参数非法	初始化关节，绑定到总线和节点 ID。若已初始化则先自动释放 [非实时接口]
4	<code>NrtDestroy()</code>	无	<code>void</code>	释放关节资源，从总线注销 [非实时接口]
5	<code>NrtNodeId()</code>	无	<code>uint8_t</code> — 当前绑定的 <code>node_id</code> ，未初始化返回 0	获取当前绑定的节点 ID [非实时接口]

禁止拷贝构造和拷贝赋值。

5.2 PDO 实时控制（非阻塞）

所有 `Set*Target*` 函数仅将帧入队（SPSC 无锁队列），由内部 Tx 线程异步执行 `write()`，不阻塞调用者。`controlword` 由 SDK 内部缓存自动携带。

典型调用顺序：① `bus.RtSendSync(counter)` → ② `joint.RtGetJointStatus(st)` → ③ `joint.RtSetCspTargetPosition(pos_rad)`

序号	接口名字	参数	返回值	说明
6	<code>RtSetCspTargetPosition(target_pos_rad)</code>	<code>target_pos_rad</code> : 目标位置（ <code>const float</code> ），单位：rad	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	CSP 模式：发送目标位置（RxPDO0， <code>cob_id = 0x200 + node_id</code> ） [实时接口]
7	<code>RtSetCsvTargetVelocity(target_vel_rads)</code>	<code>target_vel_rads</code> : 目标速度（ <code>const float</code> ），单位：rad/s	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	CSV 模式：发送目标速度（RxPDO1， <code>cob_id = 0x240 + node_id</code> ） [实时接口]
8	<code>RtSetCstTargetCurrent(target_cur_a)</code>	<code>target_cur_a</code> : 目标转矩电流（ <code>const float</code> ），单位：A	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	CST 模式：发送目标转矩电流（RxPDO2， <code>cob_id = 0x280 + node_id</code> ） [实时接口]

序号	接口名字	参数	返回值	说明
9	<code>RtSetPpTargetPosition(target_pos_rad, profile_vel_rads, profile_acc_radss, profile_dec_radss)</code>	<code>target_pos_rad</code> : 目标位置 (const float)，单位: rad <code>profile_vel_rads</code> : 轮廓速度 (const float)，单位: rad/s <code>profile_acc_radss</code> : 轮廓加速度 (const float)，单位: rad/s² <code>profile_dec_radss</code> : 轮廓减速度 (const float)，单位: rad/s²	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	PP 模式：发送轮廓位置目标 (RxPDO3, <code>cob_id = 0x2C0 + node_id</code>) [实时接口]

5.3 状态读取（非阻塞）

序号	接口名字	参数	返回值	说明
10	<code>RtGetJointStatus(out)</code>	<code>out</code> : [out] 输出状态结构体 (JointStatus&)	<code>int</code> — <code>CanEvoError::kOk</code> 成功 (有新数据)； <code>kNotInitialized</code> 未初始化	获取最新关节状态，从 TxPDO0 缓存读取 [实时接口]
11	<code>RtGetEmcy(out_fault)</code>	<code>out_fault</code> : [out] 输出故障码枚举 (CanEvoFault&)	<code>bool</code> — <code>true</code> : 有新的 EMCY 报文； <code>false</code> : 无	获取最新 EMCY 故障码，从 EMCY 队列消费。EMCY 是事件驱动的实时通知 (<code>cob_id = 0x80 + node_id</code>) [实时接口]

5.4 控制字控制（非阻塞）

修改内部 controlword 缓存，下次 PDO 帧发送时自动生效。所有函数均为非阻塞操作。

序号	接口名字	参数	返回值	说明
12	<code>RtEnable(mode)</code>	<code>mode</code> : 目标模式 (const CanEvoMode)	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	伺服使能并设置工作模式 (设置控制字 bit1 = 1 和 bit12~bit15) [实时接口]
13	<code>RtDisable()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	伺服失能 (先切换到 CSP 模式，再设置控制字 bit1 = 0)。非阻塞，修改内部 controlword 缓存，下次 PDO 发送时生效 [实时接口]
14	<code>RtClearFault()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	清除故障 (触发控制字 bit0 单周期脉冲，下次 PDO 发送时 bit0=1，之后自动清零) [实时接口]

序号	接口名字	参数	返回值	说明
16	<code>RtEstop()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	触发急停（设置控制字 <code>bit2 = 1</code> ）[实时接口]
17	<code>RtClearEstop()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	清除急停（设置控制字 <code>bit2 = 0</code> ）[实时接口]

5.5 状态查询辅助（非阻塞）

内部从 `TxPDO0` 缓存读取状态字并解析，无需传入状态字参数。

序号	接口名字	参数	返回值	说明
18	<code>RtGetServoState()</code>	无	<code>CanEvoServoState</code> — 伺服状态机枚举值	获取伺服状态机状态（解析状态字 <code>bit0~bit3</code> ）[实时接口]
19	<code>RtGetCurrentMode()</code>	无	<code>CanEvoMode</code> — 当前工作模式枚举值	获取当前工作模式（解析状态字 <code>bit12~bit15</code> ）[实时接口]
20	<code>RtIsRunning()</code>	无	<code>bool</code> — <code>true</code> : 伺服使能运行中	检查是否处于运行状态（状态 = <code>kRunning</code> ）[实时接口]
21	<code>RtHasWarning()</code>	无	<code>bool</code> — <code>true</code> : 存在警告	检查是否有警告（状态字 <code>bit4</code> ）[实时接口]

5.6 SDO 设备信息（阻塞，Index 0x00）

所有 `get` 接口直接返回结果值，SDO 通信失败时返回 `0` / 默认值。

序号	接口名字	参数	返回值	说明
22	<code>NrtGetProtocolVersion()</code>	无	<code>uint16_t</code> — 协议版本	<code>0x00/0x01</code> 读取协议版本 (R) [非实时接口]
23	<code>NrtGetModuleType()</code>	无	<code>uint32_t</code> — 关节代号	<code>0x00/0x02</code> 读取关节代号 (R) [非实时接口]
24	<code>NrtGetVendorCode()</code>	无	<code>uint32_t</code> — 厂商代号	<code>0x00/0x03</code> 读取厂商代号 (R) [非实时接口]
25	<code>NrtGetGlobalId(id)</code>	<code>id</code> : [out] 输出数组 (<code>uint32_t[4]</code>)	<code>int</code> — <code>CanEvoError::kOk</code> 成功	<code>0x00/0x04~0x07</code> 读取关节全球 ID (<code>4×uint32</code> , R) [非实时接口]
26	<code>NrtGetFaultCode()</code>	无	<code>CanEvoFault</code> — 故障码枚举值, <code>kNone</code> 表示无故障	<code>0x00/0x08</code> 读取故障代码 (SDO 阻塞轮询, R) [非实时接口]
27	<code>NrtGetWarnCode()</code>	无	<code>CanEvoWarn</code> — 警告码枚举值, <code>kNone</code> 表示无警告	<code>0x00/0x09</code> 读取警告代码 (SDO 阻塞轮询, R) [非实时接口]

序号	接口名字	参数	返回值	说明
28	<code>NrtGetDiagInfo()</code>	无	<code>uint16_t</code> — 诊断信息	0x00/0x0A 读取诊断信息 (R) [非实时接口]
29	<code>NrtGetCanBaud()</code>	无	<code>uint16_t</code> — 波特率编码	0x00/0x0B 读取 CAN 波特率编码 (RW) [非实时接口]
30	<code>NrtSetCanBaud(baud)</code>	<code>baud</code> : 波特率编码 (const <code>uint16_t</code>)	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x00/0x0B 写入 CAN 波特率编码 — 保存参数 + 重启后生效 [非实时接口]
31	<code>NrtGetCanId()</code>	无	<code>uint16_t</code> — CAN ID (1~62)	0x00/0x0C 读取 CAN ID (RW) [非实时接口]
32	<code>NrtSetCanId(can_id)</code>	<code>can_id</code> : CAN ID (const <code>uint16_t</code> , 1~62)	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x00/0x0C 写入 CAN ID — 保存参数 + 重启后生效 [非实时接口]
33	<code>NrtGetFwVersion()</code>	无	<code>uint16_t</code> — 固件版本	0x00/0x0D 读取固件版本 (R) [非实时接口]
34	<code>NrtGetHwVersion()</code>	无	<code>uint16_t</code> — 硬件版本	0x00/0x0E 读取硬件版本 (R) [非实时接口]
35	<code>NrtGetSyncPeriod()</code>	无	<code>uint16_t</code> — 同步周期, 单位: us	0x00/0x0F 读取同步周期 (RW) [非实时接口]
36	<code>NrtSetSyncPeriod(us)</code>	<code>us</code> : 同步周期 (const <code>uint16_t</code>), 单位: us	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x00/0x0F 写入同步周期 [非实时接口]
37	<code>NrtGetCommTimeout()</code>	无	<code>uint16_t</code> — 通信超时时间, 单位: ms	0x00/0x10 读取通信超时时间 (RW) [非实时接口]
38	<code>NrtSetCommTimeout(ms)</code>	<code>ms</code> : 超时时间 (const <code>uint16_t</code>), 单位: ms	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x00/0x10 写入通信超时时间 [非实时接口]

5.7 SDO 关节参数（阻塞，Index 0x01）

位置、速度、加速度相关接口的公开单位为 rad / rad/s / rad/s²，SDK 内部自动完成与协议层 deg / rpm 的转换。

序号	接口名字	参数	返回值	说明
39	<code>NrtGetGearRatio(num, den)</code>	<code>num</code> : [out] 分子 (<code>uint16_t&</code>) <code>den</code> : [out] 分母 (<code>uint16_t&</code>)	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x01/0x00~0x01 获取减速比 (R)， 减速比 = num / den [非实时接口]
40	<code>NrtGetMaxSpeed()</code>	无	<code>float</code> — 最大速度, 单位: rad/s	0x01/0x02 获取最大速度 (RW) [非实时接口]
41	<code>NrtSetMaxSpeed(rads)</code>	<code>rads</code> : 最大速度 (const float), 单位: rad/s	<code>int</code> — <code>CanEvoError::kOk</code> 成功	0x01/0x02 设置最大速度 [非实时接口]

序号	接口名字	参数	返回值	说明
42	<code>NrtGetMaxAccel()</code>	无	<code>float</code> — 最大加速度，单位：rad/s²	0x01/0x03 获取最大加速度（RW） [非实时接口]
43	<code>NrtSetMaxAccel(radss)</code>	<code>radss</code> : 最大加速度（const float），单位：rad/s²	<code>int</code> — CanEvoError::kOk 成功	0x01/0x03 设置最大加速度 [非实时接口]
44	<code>NrtGetMaxDecel()</code>	无	<code>float</code> — 最大减速度，单位：rad/s²	0x01/0x04 获取最大减速度（RW） [非实时接口]
45	<code>NrtSetMaxDecel(radss)</code>	<code>radss</code> : 最大减速度（const float），单位：rad/s²	<code>int</code> — CanEvoError::kOk 成功	0x01/0x04 设置最大减速度 [非实时接口]
46	<code>NrtGetEstopDecel()</code>	无	<code>float</code> — 急停减速度，单位：rad/s²	0x01/0x05 获取急停减速度（RW） [非实时接口]
47	<code>NrtSetEstopDecel(radss)</code>	<code>radss</code> : 急停减速度（const float），单位：rad/s²	<code>int</code> — CanEvoError::kOk 成功	0x01/0x05 设置急停减速度 [非实时接口]
48	<code>NrtGetMaxPos()</code>	无	<code>float</code> — 最大位置，单位：rad	0x01/0x06 获取最大位置（RW） [非实时接口]
49	<code>NrtSetMaxPos(rad)</code>	<code>rad</code> : 最大位置（const float），单位：rad	<code>int</code> — CanEvoError::kOk 成功	0x01/0x06 设置最大位置 [非实时接口]
50	<code>NrtGetMinPos()</code>	无	<code>float</code> — 最小位置，单位：rad	0x01/0x07 获取最小位置（RW） [非实时接口]
51	<code>NrtSetMinPos(rad)</code>	<code>rad</code> : 最小位置（const float），单位：rad	<code>int</code> — CanEvoError::kOk 成功	0x01/0x07 设置最小位置 [非实时接口]
52	<code>NrtSetMechZero()</code>	无	<code>int</code> — CanEvoError::kOk 成功	0x01/0x08 设置机械零点（W，写入 0x0001 触发）。建议在伺服失能状态下执行 [非实时接口]
53	<code>NrtGetMechZeroOk()</code>	无	<code>uint16_t</code> — 1=成功	0x01/0x09 读取机械零点设置成功标志（R） [非实时接口]
54	<code>NrtSetPosLimitEnable(enable)</code>	<code>enable</code> : 使能标志（const uint16_t, 1=使能/0=禁止）	<code>int</code> — CanEvoError::kOk 成功	0x01/0x0A 位置限制使能（W） [非实时接口]
55	<code>NrtGetPosLimitStatus()</code>	无	<code>uint16_t</code> — 0=未使能, 1=已使能	0x01/0x0B 读取位置限制状态（R） [非实时接口]

序号	接口名字	参数	返回值	说明
56	<code>NrtSetBrakeControl(val)</code>	<code>val</code> : 抱闸控制字 (const uint16_t) kCanevoBrakeOpen(0xAA55)= 打开 kCanevoBrakeClose(0xBB66)= 关闭	<code>int</code> — CanEvoError::kOk 成功	0x01/0x0C 抱闸控制 (W) [非实时接口]
57	<code>NrtGetBrakeStatus()</code>	无	<code>uint16_t</code> — 0= 关闭, 1=打开	0x01/0x0D 读取抱闸状态 (R) [非实时接口]
58	<code>NrtSaveParams()</code>	无	<code>int</code> — CanEvoError::kOk 成功	0x01/0xF0 保存参数 (W, 写入 0x0001 触发)。建议在伺服失能状态下执行 [非实时接口]
59	<code>NrtGetSaveParamsOk()</code>	无	<code>uint16_t</code> — 1= 成功	0x01/0xF1 读取参数保存成功标志 (R) [非实时接口]

5.8 SDO 控制器参数（阻塞，Index 0x07）

全部为 uint16 类型，RW 权限。

序号	接口名字	参数	返回值	说明
60	<code>NrtGetCtrlParam(sub)</code>	<code>sub</code> : 子索引 (const uint8_t, 0x00~0x0A)	<code>uint16_t</code> — 参数值, 失败返回 0	0x07/sub 通用读取控制器参数 [非实时接口]
61	<code>NrtSetCtrlParam(sub, val)</code>	<code>sub</code> : 子索引 (const uint8_t, 0x00~0x0A) <code>val</code> : 写入值 (const uint16_t)	<code>int</code> — CanEvoError::kOk 成功	0x07/sub 通用写入控制器参数 [非实时接口]

5.9 SDO 编码器参数（阻塞，Index 0x08）

序号	接口名字	参数	返回值	说明
62	<code>NrtGetMotorEncRes()</code>	无	<code>uint32_t</code> — 分辨率, 单位: Pulse/Rev	0x08/0x00 读取电机轴编码器分辨率 (R) [非实时接口]
63	<code>NrtGetOutputEncRes()</code>	无	<code>uint32_t</code> — 分辨率, 单位: Pulse/Rev	0x08/0x01 读取输出轴编码器分辨率 (R) [非实时接口]

5.10 SDO PP 模式轮廓参数（阻塞，Index 0x21）

一次性设置 PP 模式的所有轮廓参数，通过 SDO 写入 0x21/0x02, 0x21/0x07, 0x21/0x08, 0x21/0x09。

序号	接口名字	参数	返回值	说明
----	------	----	-----	----

序号	接口名字	参数	返回值	说明
64	<code>NrtSetPpTargetPosition(target_pos_rad, profile_vel_rads, profile_acc_radss, profile_dec_radss)</code>	<code>target_pos_rad</code> : 目标位置 (const float) , 单位: rad <code>profile_vel_rads</code> : 轮廓速度 (const float) , 单位: rad/s <code>profile_acc_radss</code> : 轮廓加速度 (const float) , 单位: rad/s² <code>profile_dec_radss</code> : 轮廓减速度 (const float) , 单位: rad/s²	<code>int</code> — <code>CanEvoError::kOk</code> 成功	一次性设置 PP 模式轮廓参数 (0x21/0x02, 0x21/0x07, 0x21/0x08, 0x21/0x09) [非实时接口]

5.11 SDO 关节实际状态（阻塞，Index 0x20，只读）

读取关节的实际运行状态量，所有接口均为只读，用于诊断和标定。

序号	接口名字	参数	返回值	说明
65	<code>NrtGetActualPosition()</code>	无	<code>float</code> — 实际位置, 单位: rad	0x20/0x00 读取实际位置 (R) [非实时接口]
66	<code>NrtGetActualVelocity()</code>	无	<code>float</code> — 实际速度, 单位: rad/s	0x20/0x01 读取实际速度 (R) [非实时接口]
67	<code>NrtGetActualCurrent()</code>	无	<code>float</code> — 实际转矩电流, 单位: A	0x20/0x02 读取实际转矩电流 (R) [非实时接口]
68	<code>NrtGetActualAcceleration()</code>	无	<code>float</code> — 实际加速度, 单位: rad/s²	0x20/0x03 读取实际加速度 (R) [非实时接口]
69	<code>NrtGetActualTorque()</code>	无	<code>float</code> — 实际转矩, 单位: Nm	0x20/0x04 读取实际转矩 (R) [非实时接口]
70	<code>NrtGetPositionTrackingError()</code>	无	<code>float</code> — 位置跟踪误差, 单位: rad	0x20/0x05 读取位置跟踪误差 (R) [非实时接口]
71	<code>NrtGetBusVoltage()</code>	无	<code>float</code> — 母线电压, 单位: V	0x20/0x06 读取母线电压 (R, raw×0.01→V) [非实时接口]
72	<code>NrtGetPcbTemperature()</code>	无	<code>float</code> — PCB 温度, 单位: °C	0x20/0x07 读取 PCB 温度 (R, raw×0.1→°C) [非实时接口]
73	<code>NrtGetMotorTemperature()</code>	无	<code>float</code> — 电机温度, 单位: °C	0x20/0x08 读取电机温度 (R, raw×0.1→°C) [非实时接口]
74	<code>NrtGetGearboxTemperature()</code>	无	<code>float</code> — 减速器温度, 单位: °C	0x20/0x09 读取减速器温度 (R, raw×0.1→°C) [非实时接口]
75	<code>NrtGetElectricalAngle()</code>	无	<code>float</code> — 电角度, 单位: rad	0x20/0x0A 读取电角度 (R, raw×0.1 deg→rad) [非实时接口]

5.12 SDO 控制字操作（阻塞）

通过 SDO 直接读写控制字（0x21/0x00），每次只修改相应位，不改变其他位。操作完成后控制字立即生效，无需等待 PDO 发送。所有函数均为阻塞调用，会等待从站应答或超时。

序号	接口名字	参数	返回值	说明
76	<code>NrtEnable(mode)</code>	<code>mode</code> : 目标模式 (const <code>CanEvoMode</code>)	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	SDO 方式伺服使能并设置工作模式（设置控制字 bit1 = 1 和 bit12~bit15）。通过 SDO 修改控制字，操作完成后控制字立即生效，无需等待 PDO 发送 [非实时接口]
77	<code>NrtDisable()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	SDO 方式伺服失能（先切换到 CSP 模式，再设置控制字 bit1 = 0）。通过 SDO 修改控制字，先设置模式为 CSP (bit12~bit15)，再失能 (bit1=0)，操作完成后控制字立即生效，无需等待 PDO 发送 [非实时接口]
79	<code>NrtClearFault()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	SDO 方式清除故障（触发控制字 bit0 单周期脉冲）。通过 SDO 发送单周期脉冲(bit0=1然后自动清0)，仅在故障状态时有效 [非实时接口]
80	<code>NrtEstop()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	SDO 方式触发急停（设置控制字 bit2 = 1）。通过 SDO 修改控制字 bit2=1，每次只修改急停位，不改变其他位 [非实时接口]
81	<code>NrtClearEstop()</code>	无	<code>int</code> — <code>CanEvoError::kOk</code> 成功； <code>kNotInitialized</code> 未初始化	SDO 方式清除急停（设置控制字 bit2 = 0）。通过 SDO 修改控制字 bit2=0，每次只修改急停位，不改变其他位 [非实时接口]

与 Rt 接口的区别：

- **Rt 接口**（如 `RtEnable(mode)`）：修改内部 controlword 缓存（非阻塞），下次 PDO 帧发送时自动携带生效，适用于实时控制循环
- **Nrt 接口**（如 `NrtEnable(mode)`）：通过 SDO 直接读写控制字（阻塞），操作完成后立即生效，适用于初始化/配置阶段或需要立即生效的场景

6. PDO 映射方案

6.1 RxPDO（主站 → 从站，控制指令）

PDO	cob_id	数据内容	长度
RxPDO0	0x200 + node_id	controlword(u16) + 目标位置(float, deg)	8B
RxPDO1	0x240 + node_id	controlword(u16) + 目标速度(float, rpm)	8B
RxPDO2	0x280 + node_id	controlword(u16) + 目标电流(float, A)	8B
RxPDO3	0x2C0 + node_id	controlword(u16) + 目标位置(float) + 轮廓速度(float) + 轮廓加速(float) + 轮廓减速(float)	20B

6.2 TxPDO（从站 → 主站，状态反馈）

PDO	cob_id	数据内容	长度
TxPDO0	0x400 + node_id	statusword(u16) + 位置(float) + 速度(float) + 电流(float) + 加速度(float) + 母线电压(u16) + PCB温度(i16) + 电机温度(i16)	24B

6.3 其他

帧类型	cob_id	说明
SYNC	0x03F	同步广播帧，由 Bus 发送
EMCY	0x080 + node_id	紧急报文（故障时从站主动上报）
SDO Request	0x780 + node_id	SDO 请求帧（主站 → 从站）
SDO Response	0x7C0 + node_id	SDO 应答帧（从站 → 主站）

7. 对象字典索引参考

对象索引	子索引	名称	类型	访问	接口
0x00	0x01	协议版本	uint16	R	NrtGetProtocolVersion()
0x00	0x02	关节代号	uint32	R	NrtGetModuleType()
0x00	0x03	厂商代号	uint32	R	NrtGetVendorCode()
0x00	0x04~0x07	全球ID	uint32×4	R	NrtGetGlobalId()
0x00	0x08	故障代码	uint16	R	NrtGetFaultCode()
0x00	0x09	警告代码	uint16	R	NrtGetWarnCode()
0x00	0x0A	诊断信息	uint16	R	NrtGetDiagInfo()
0x00	0x0B	CAN波特率	uint16	RW	NrtGetCanBaud() / NrtSetCanBaud()
0x00	0x0C	CAN ID	uint16	RW	NrtGetCanId() / NrtSetCanId()
0x00	0x0D	固件版本	uint16	R	NrtGetFwVersion()
0x00	0x0E	硬件版本	uint16	R	NrtGetHwVersion()
0x00	0x0F	同步周期	uint16	RW	NrtGetSyncPeriod() / NrtSetSyncPeriod()
0x00	0x10	通信超时	uint16	RW	NrtGetCommTimeout() / NrtSetCommTimeout()
0x01	0x00~0x01	减速比	uint16×2	R	NrtGetGearRatio()
0x01	0x02	最大速度	float	RW	NrtGetMaxSpeed() / NrtSetMaxSpeed()
0x01	0x03	最大加速度	float	RW	NrtGetMaxAccel() / NrtSetMaxAccel()
0x01	0x04	最大减速度	float	RW	NrtGetMaxDecel() / NrtSetMaxDecel()
0x01	0x05	急停减速度	float	RW	NrtGetEstopDecel() / NrtSetEstopDecel()
0x01	0x06	最大位置	float	RW	NrtGetMaxPos() / NrtSetMaxPos()
0x01	0x07	最小位置	float	RW	NrtGetMinPos() / NrtSetMinPos()
0x01	0x08	设置机械零点	uint16	W	NrtSetMechZero()

对象索引	子索引	名称	类型	访问	接口
0x01	0x09	机械零点标志	uint16	R	<code>NrtGetMechZeroOk()</code>
0x01	0x0A	位置限制使能	uint16	W	<code>NrtSetPosLimitEnable()</code>
0x01	0x0B	位置限制状态	uint16	R	<code>NrtGetPosLimitStatus()</code>
0x01	0x0C	抱闸控制	uint16	W	<code>NrtSetBrakeControl()</code>
0x01	0x0D	抱闸状态	uint16	R	<code>NrtGetBrakeStatus()</code>
0x01	0xF0	保存参数	uint16	W	<code>NrtSaveParams()</code>
0x01	0xF1	保存参数标志	uint16	R	<code>NrtGetSaveParamsOk()</code>
0x07	0x00~0x0A	控制器参数	uint16	RW	<code>NrtGetCtrlParam()</code> / <code>NrtSetCtrlParam()</code>
0x08	0x00	电机轴编码器分辨率	uint32	R	<code>NrtGetMotorEncRes()</code>
0x08	0x01	输出轴编码器分辨率	uint32	R	<code>NrtGetOutputEncRes()</code>
0x20	0x00	实际位置	float	R	<code>NrtGetActualPosition()</code>
0x20	0x01	实际速度	float	R	<code>NrtGetActualVelocity()</code>
0x20	0x02	实际转矩电流	float	R	<code>NrtGetActualCurrent()</code>
0x20	0x03	实际加速度	float	R	<code>NrtGetActualAcceleration()</code>
0x20	0x04	实际转矩	float	R	<code>NrtGetActualTorque()</code>
0x20	0x05	位置跟踪误差	float	R	<code>NrtGetPositionTrackingError()</code>
0x20	0x06	母线电压	uint16	R	<code>NrtGetBusVoltage()</code>
0x20	0x07	PCB温度	int16	R	<code>NrtGetPcbTemperature()</code>
0x20	0x08	电机温度	int16	R	<code>NrtGetMotorTemperature()</code>
0x20	0x09	减速器温度	int16	R	<code>NrtGetGearboxTemperature()</code>
0x20	0x0A	电角度	uint16	R	<code>NrtGetElectricalAngle()</code>
0x21	0x00	控制字	uint16	RW	<code>NrtEnable(mode)</code> / <code>NrtDisable()</code> / <code>NrtClearFault()</code> / <code>NrtEstop()</code> / <code>NrtClearEstop()</code> (SDO方式, 阻塞) 或通过 <code>RtEnable(mode)</code> / <code>RtDisable()</code> 等 (PDO方式, 非阻塞)
0x21	0x02	PP模式目标位置	float	RW	<code>NrtSetPpTargetPosition()</code> (一次性设置所有PP参数)

对象索引	子索引	名称	类型	访问	接口
0x21	0x07	轮廓速度	float	RW	<code>NrtSetPpTargetPosition()</code> (一次性设置所有PP参数)
0x21	0x08	轮廓加速度	float	RW	<code>NrtSetPpTargetPosition()</code> (一次性设置所有PP参数)
0x21	0x09	轮廓减速度	float	RW	<code>NrtSetPpTargetPosition()</code> (一次性设置所有PP参数)

8. 典型用法

```
#include "modi_joint_canevo.h"

int main() {
    // 1. 创建并打开总线
    modi_bus_canevo bus;
    bus.Open("can0");

    // 2. 创建并初始化关节
    modi_joint_canevo j1, j2;
    j1.NrtInit(bus, 1);
    j2.NrtInit(bus, 2);

    // 3. 配置模式并使能
    j1.RtEnable(CanEvoMode::kCsp);
    j2.RtEnable(CanEvoMode::kCsp);

    // 4. 实时控制循环
    uint8_t cnt = 0;
    bool running = true;
    while (running) {
        bus.RtSendSync(cnt++);

        JointStatus st;
        j1.RtGetJointStatus(st);
        // 使用 st.actual_pos_rad, st.actual_vel_rads 等...

        j1.RtSetCspTargetPosition(1.57f); // 目标位置  $\pi/2$  rad
        j2.RtSetCspTargetPosition(3.14f); // 目标位置  $\pi$  rad

        // 检查故障
        CanEvoFault fault;
        if (j1.RtGetEmcy(fault)) {
            // 处理 EMCY 故障...
        }
    }

    // 5. 失能并释放
    j1.RtDisable();
    j2.RtDisable();

    j1.NrtDestroy();
    j2.NrtDestroy();
    bus.Close();
    return 0;
}
```

9. 注意事项

1. **实时接口（Rt 前缀）**：所有以 `Rt` 开头的接口为实时接口，包括 PDO 控制、状态读取、控制字控制等。这些接口非阻塞，通过 SPSC 无锁队列 + 专用 Tx 线程发送，保证实时性。典型接口：`RtSetCspTargetPosition()`、`RtGetJointStatus()`、`RtEnable()`、`RtSendSync()` 等。
2. **非实时接口（Nrt 前缀）**：所有以 `Nrt` 开头的接口为非实时接口，包括生命周期管理、SDO 配置等。SDO 接口为同步阻塞调用，会等待从站应答或超时（默认 100ms），不应在实时控制循环中调用。典型接口：`NrtInit()`、`NrtDestroy()`、`NrtGetProtocolVersion()`、`NrtSetMaxSpeed()` 等。
3. **控制字管理**：`RtEnable(mode)`、`RtDisable()`、`RtClearFault()` 等函数修改内部 controlword 缓存（非阻塞），下次 PDO 帧发送时自动携带生效。用户无需手动管理 controlword。
4. **SYNC 广播**：每个控制周期调用一次 `bus.RtSendSync(counter)`，counter 0~255 循环递增。SYNC 使用独立 socket fd 发送，避免与 PDO 争抢锁。
5. **故障检测**：有两种方式获取故障信息：
 - `RtGetEmcy()`：非阻塞，从 EMCY 队列消费，适用于实时循环中的事件驱动检测
 - `NrtGetFaultCode()`：SDO 阻塞轮询，适用于非实时的诊断查询
6. **单位转换**：所有面向用户的位置/速度/加速度接口均使用 rad 系单位，SDK 内部自动完成与协议层 deg / rpm 的转换。
7. **线程安全**：实时接口（Rt 前缀）建议在单线程实时循环中调用（减少周期抖动）；非实时接口（Nrt 前缀）同一 Joint 不建议并发（SDK 内部 per-joint 串行化）。
8. **生命周期**：必须先 `bus.Open()` → `joint.NrtInit()` → 使用 → `joint.NrtDestroy()` → `bus.Close()`。析构函数会自动调用 `NrtDestroy()`。